LAB 3: NumPy Advanced Operations and Data Analysis

Procedure

Step 1:

Data Loading:
NumPy, Pandas, and Matplotlib are imported.

Dataset:
A CSV file 'AAPL_stock_data.csv' is loaded with 'Date' as index.

Extraction:
The 'Close' column is converted into a NumPy array.

Output:
The shape (1258,) and sample values of closing prices are displayed.

```python
import numpy as np
import pandas as pd
import time
import matplotlib.pyplot as plt

# Load the data from the CSV file
df = pd.read_csv('AAPL_stock_data.csv', index_col='Date', parse_dates=True)

# Extract the 'Close' prices as a 1D NumPy array
close_prices = df['Close'].values
print(f"Close Prices (1D Array): {close_prices.shape}")
print(close_prices[:5])
```

```
Close Prices (1D Array): (1258,)
[37.50371933 33.76808548 35.2096138  35.13124466 35.80096054]
```

Figure 3.1

Step 2:

Array Transformation:
The 1D array is reshaped into a 2D array where each row represents 5 trading days.

Further Structuring:
A 3D array is created representing months, weeks, and days.

Output:
Shapes of 2D (251,5) and 3D arrays are printed.

```python
# 1D Array (Already created above)
a_1d = close_prices

# 2D Array: Let's create a 2D array where each row represents a week of trading (5 days)
# We'll trim the data to be a multiple of 5
num_weeks = len(a_1d) // 5
a_2d = a_1d[:num_weeks * 5].reshape(num_weeks, 5)
print(f"2D Array (Weeks x Days): {a_2d.shape}")

# 3D Array: Let's create a 3D array where each 'layer' represents a month (4 weeks per month)
num_months = num_weeks // 4
a_3d = a_2d[:num_months * 4].reshape(num_months, 4, 5)
print(f"3D Array (Months x Weeks x Days): {a_3d.shape}")
```

```
2D Array (Weeks x Days): (251, 5)
3D Array (Months x Weeks x Days): (62, 4, 5)
```

Figure 3.2

Step 3:

Indexing:
Accesses the price of day 1 in week 2 using a_2d[1,0].

Slicing:
Extracts first 4 weeks of data.

Reshaping:
Flattens 3D array back to 1D.

Output:
Displays selected value and shapes.

```python
# Indexing: Accessing the price on the first day of the second week
print(f"Price on day 1 of week 2: {a_2d[1, 0]}")

# Slicing: Getting all prices for the first 4 weeks
first_4_weeks = a_2d[:4, :]
print(f"First 4 weeks shape: {first_4_weeks.shape}")

# Reshaping: Flattening the 3D array back to 1D
flattened_3d = a_3d.reshape(-1)
print(f"Flattened 3D array shape: {flattened_3d.shape}")
```

```
Price on day 1 of week 2: 36.40891647338867
First 4 weeks shape: (4, 5)
Flattened 3D array shape: (1240,)
```

Figure 3.3

Step 4:

Log Returns:
Calculates logarithmic returns using np.log.

Normalization:
Divides prices by mean price.

Broadcasting:
Subtracts weekly mean from daily prices.

Output:
Displays first values and resulting shapes.

```python
# Element-wise: Calculating daily returns (log returns)
log_returns = np.log(a_1d[1:] / a_1d[:-1])
print(f"First 5 log returns: {log_returns[:5]}")

# Broadcasting: Normalizing the prices by the mean price
mean_price = np.mean(a_1d)
normalized_prices = a_1d / mean_price
print(f"First 5 normalized prices: {normalized_prices[:5]}")

# Broadcasting with 2D array: Subtracting the weekly mean from each day's price
weekly_means = a_2d.mean(axis=1, keepdims=True)
centered_weekly_prices = a_2d - weekly_means
print(f"Centered weekly prices shape: {centered_weekly_prices.shape}")
```

```
First 5 log returns: [-0.10492397 0.04180302 -0.00222827 0.01888383 0.01683898]
First 5 normalized prices: [0.31170285 0.28065506 0.29263596 0.29198462 0.29755079]
Centered weekly prices shape: (251, 5)
```

Figure 3.4

Step 5:

Statistical Analysis:
Calculates mean, median, standard deviation, and percentiles.

Output:
Mean 120.32, Median 131.29, Std Dev 46.32, Percentiles shown.

```python
print(f"Mean Price: {np.mean(a_1d):.2f}")
print(f"Median Price: {np.median(a_1d):.2f}")
print(f"Standard Deviation: {np.std(a_1d):.2f}")
print(f"25th Percentile: {np.percentile(a_1d, 25):.2f}")
print(f"75th Percentile: {np.percentile(a_1d, 75):.2f}")
```

```
Mean Price: 120.32
Median Price: 131.29
Standard Deviation: 46.32
25th Percentile: 74.62
75th Percentile: 156.59
```

Figure 3.5

Step 6:

Linear Algebra:
Computes weighted sum using dot product.

Matrix Operation:
Projects data using matrix multiplication.

Output:
Shows weighted values and projected shape (251,3).

```python
# Dot Product: Weighted sum of prices (e.g., for a portfolio)
weights = np.linspace(0.1, 1.0, 5) / np.sum(np.linspace(0.1, 1.0, 5))
weighted_weekly_prices = np.dot(a_2d, weights)
print(f"Weighted weekly prices (first 5): {weighted_weekly_prices[:5]}")

# Matrix Multiplication: Projecting the weekly prices onto a new basis
projection_matrix = np.random.rand(5, 3)
projected_prices = np.matmul(a_2d, projection_matrix)
print(f"Projected prices shape: {projected_prices.shape}")
```

```
Weighted weekly prices (first 5): [35.31562271 36.13196387 36.71328732 37.79337328 40.94942689]
Projected prices shape: (251, 3)
```

Figure 3.6

Step 7:

Random Simulation:
Generates normal distributed returns.

Sampling:
Randomly samples from computed log returns.

Output:
Displays mean values of simulated and sampled data.

```python
# Generating random normal data (simulating stock returns)
simulated_returns = np.random.normal(loc=0.0005, scale=0.01, size=1000)
print(f"Simulated returns mean: {np.mean(simulated_returns):.5f}")

# Sampling from our existing log returns
sampled_returns = np.random.choice(log_returns, size=100, replace=True)
print(f"Sampled returns mean: {np.mean(sampled_returns):.5f}")
```

```
Simulated returns mean: 0.00003
Sampled returns mean: 0.00056
```

Figure 3.7

Step 8:

Performance Comparison:
Adds two large lists using Python loops and NumPy arrays.

Timing:
Measures execution time for both methods.

Result:
NumPy operation is significantly faster than Python lists.

```python
size = 1000000
list_a = list(range(size))
list_b = list(range(size))
array_a = np.arange(size)
array_b = np.arange(size)

# Python List Addition
start_time = time.time()
list_sum = [a + b for a, b in zip(list_a, list_b)]
python_time = time.time() - start_time

# NumPy Array Addition
start_time = time.time()
array_sum = array_a + array_b
numpy_time = time.time() - start_time

print(f"Python List Addition Time: {python_time:.5f} seconds")
print(f"NumPy Array Addition Time: {numpy_time:.5f} seconds")
print(f"NumPy is {python_time / numpy_time:.2f}x faster than Python lists for this operation.")
```

```
Python List Addition Time: 0.12414 seconds
NumPy Array Addition Time: 0.00358 seconds
NumPy is 34.64x faster than Python lists for this operation.
```

Figure 3.8